

[SEE THE SITE](#)

# Case Study (event Finder Pwa)

## MeetYouApp

Designed By  
Houssni Msehel

# Project Overview

Meet is a responsive, installable Progressive Web App (PWA) that helps users find and explore events happening in their city. It integrates with the Google Calendar API to fetch live event data, allows users to filter by city, control the number of events displayed, and view trends through interactive charts. The app remains functional even when offline and supports accessibility best practices.

## Key objectives:

- Enable users to easily browse and search for upcoming events in various cities.
- Provide a seamless, app-like experience on mobile and desktop.
- Support offline access through service worker caching.
- Visualize event data for quick insights



# Goals

- Develop a serverless, responsive PWA using modern web technologies.
- Implement Google OAuth authentication for secure data access.
- Provide users with intuitive event search and filtering tools.
- Allow offline usage and installation on any device.
- Visualize data insights such as event distribution by city.

# Features

- ✓ City-based event search
- ✓ Adjustable number of displayed events
- ✓ Expandable/collapsible event details
- ✓ Charts for events per city and genre
- ✓ Offline caching & PWA installation support
- ✓ Accessible UI with keyboard and ARIA support

# Target

- **Event enthusiasts and travelers looking for things to do.**
- **Professionals discovering tech or cultural meetups.**
- **Students exploring educational or social gatherings.**



## Duration

Creating the client-side took about twice as long as the server-side, as I wanted to develop a strong understanding of how React and Redux work together to achieve the desired functionality.



## Technologies

JavaScript, React,  
Recharts, AWS  
Lambda, Google  
Cloud  
OAuth, Jest, Jest-  
Cucumber, Puppet  
eer, Bootstrap, CS  
S, HTML, Prettier, E  
SLint



## Support

Career Specialist: Lucy  
Kenzina  
Mentors: Paulo Andrade  
Tutor: Daniel Darku

# Implementation Overview

- Event fetching: Retrieves event data via Google Calendar API through an auth server proxy.
- Filtering: CitySearch component filters events dynamically.
- Dynamic display: NumberOfEvents component controls how many results render.
- Visualization: CityEventsChart and EventGenresChart provide data summaries.
- Offline support: Cached static files and last event list via service workers.
- Testing: Unit and feature tests validate UI behaviors and data flow.

I implemented a serverless backend using Node.js that securely interacts with the Google Calendar API via OAuth2 authentication.

Instead of a traditional database, the backend functions as a serverless API that handles authentication tokens and data requests on demand.

This design provides event information in JSON format and supports a scalable architecture that runs only when needed, reducing costs and improving reliability.

## Basics

I built the backend using Node.js and deployed it via the Serverless Framework, allowing the app to run on AWS Lambda.

This lightweight, on-demand setup removes the need for maintaining a dedicated server and ensures global availability.

The backend handles secure OAuth 2.0 token exchange and event data retrieval from Google Calendar.

## Deployment

After testing locally with Postman, I deployed the serverless functions using the Serverless Framework.

This automated deployment integrates directly with AWS Lambda and API Gateway, offering reliability and minimal latency for users accessing events worldwide.

## Business Logic

The backend's primary function is to:

- Authenticate users via Google OAuth 2.0
- Retrieve event data from Google Calendar
- Send the data to the client in structured JSON format

All communication between client and server is protected by HTTPS and scoped access tokens, ensuring security and privacy.

## Security

Security is handled through:

- OAuth 2.0 and Google API scopes for permission control
- HTTPS requests to protect data in transit
- Temporary access tokens that expire automatically
- CORS configuration to allow only the Meet App's client domain

This ensures safe communication and user privacy across all sessions.



## The Build

The Meet App was built with React using a modular component-based architecture.

It consumes event data from the serverless backend and dynamically renders UI updates.

The app supports offline functionality and installation on mobile or desktop through PWA service workers.



## Create Components

I developed reusable components such as:

- `<CitySearch />` for filtering by location
- `<NumberOfEvents />` for customizing results
- `<Event />` for displaying and toggling event details
- `<App />` as the main controller

Hooks and state management ensure seamless updates as users interact with the app.



## Design

I Built with React Bootstrap for responsive layout and accessibility.

I used NProgress for dynamic loading indicators and Recharts to visualize the number of events per city, providing users with clear insights at a glance.



## Final Product

The final product is a fully tested PWA that:

- Works offline with cached data
- Supports installability on any device
- Provides real-time event data via Google Calendar API
- Passes accessibility and performance audits with Lighthouse

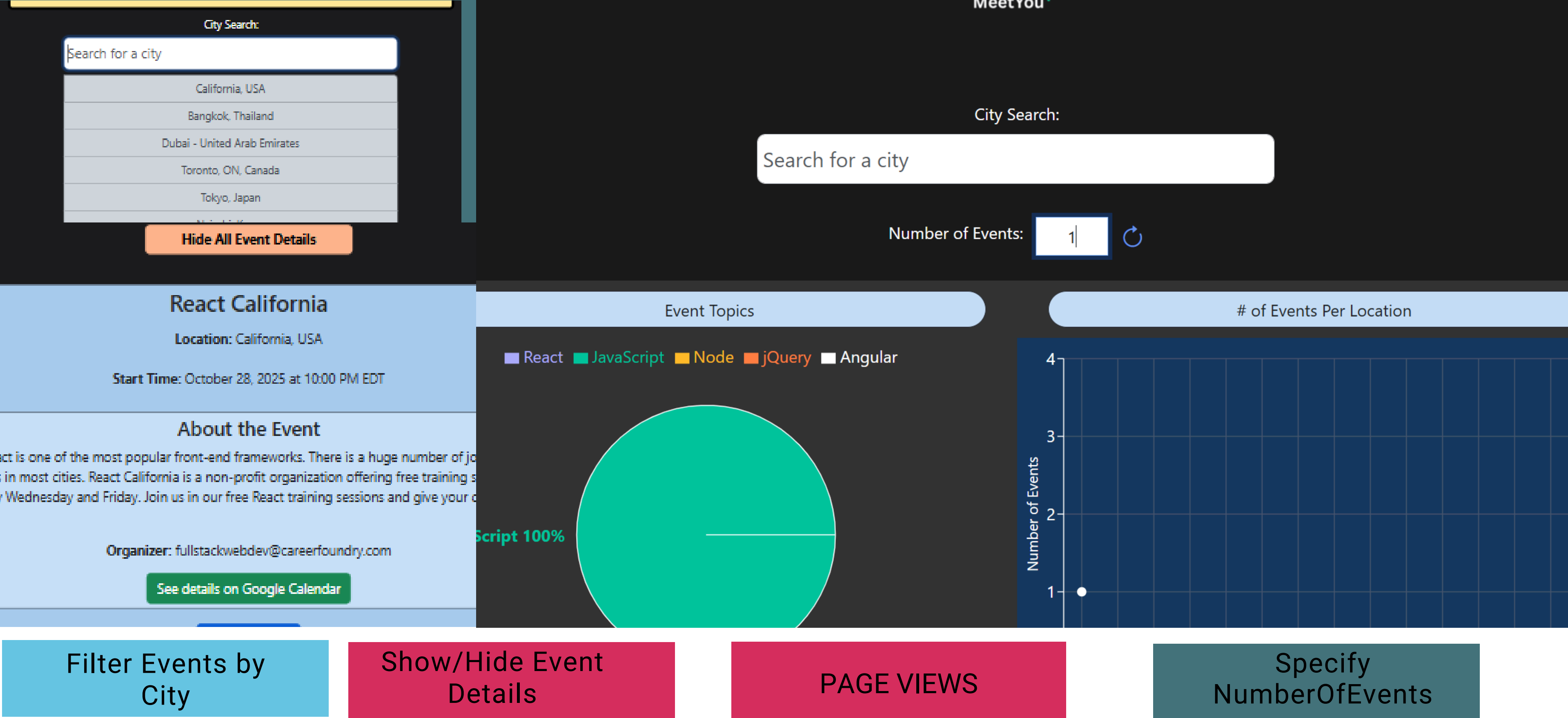
Deployed on GitHub Pages and optimized for cross-browser compatibility.

Github

# CLIENT SIDE

A React Progressive Web App (PWA) designed for finding and visualizing events around the world.

It provides a smooth, app-like experience directly in the browser, works offline, and lets users search, filter, and analyze upcoming events



Users can log in securely and access their personalized profiles within the Meet App.

From the main Event View, they can browse all upcoming events or search for events in a specific city using the built-in search feature. By selecting “Show/Hide event,” users can view a detailed event description, including date, time, location, and related information.

Additionally, the app provides insights into event series or categories available in that city, helping users discover and plan their attendance more effectively.

# Meet App – Summary

The Meet App is a serverless Progressive Web Application (PWA) built with React and integrated with the Google Calendar API. It allows users to search, filter, and visualize upcoming events around the world – even while offline.

The client side is developed using React, featuring dynamic components, responsive design with React Bootstrap, and interactive data visualizations using Recharts. Service workers enable offline access and app installation across devices.

The server side is implemented using Node.js and deployed through the Serverless Framework on AWS Lambda. It securely handles Google OAuth2 authentication, token management, and event data requests, ensuring fast and secure communication between users and the Google API.

This architecture eliminates the need for a traditional backend, resulting in a lightweight, scalable, and cost-efficient application that delivers a smooth user experience anywhere, anytime.